



Licenciatura em Engenharia Informática e Multimédia
Infraestruturas Computacionais Distribuídas

Grupo:

Gianni Floriddia – nº51945
Pedro Marques – nº51959
Rodrigo Neves – nº51452

Data: 09/06/2025

Índice:

1 - Diagrama Geral da Arquitetura	2
2 - Protocolos de Transporte	2
3 - Componentes e Funcionalidades	2
3.1 Servidor TCP (Concorrente)	2
3.2 Jogador Conectado (Concorrente).....	2
3.3 Gerir Jogadores (Iterativo)	3
3.4 JogadoresDataBase (Iterativo)	3
3.5 SalaDeEspera (Concorrente).....	3
3.6 Jogador Perfil (Iterativo)	3
3.7 ConexaoJogador (Concorrente)	3
3.8 SalaDeJogo (Concorrente)	3
3.9 Jogo e Jogador (Iterativo e Concorrente)	3
3.10 Util (Iterativo).....	3
4 Arquitetura para a WEB	3
4.1 Login.jsp	4
4.1.1 Funcionalidades da Página de Login (login.jsp)	4
4.1.2 Funcionalidades na Sala de Espera	4
4.2 Game.jsp	4
4.2.1 Implementação e Estrutura	5
4.2.2 Gestão do Estado e Comunicação	5
4.2.3 Desenho Dinâmico do Tabuleiro	5
4.2.4 Melhorias na Experiência do Jogador (UX)	5
4.3 Páginas de definições e quadro de honra.....	6
4.3.1 Página de Definições de Perfil (definicoes.jsp)	6
4.3.2 Quadro de Honra (quadro_honra.jsp)	6
5 - Estruturas de Dados Persistentes	7
6 - Protocolo de Aplicação	7
7 – Exemplos da Aplicação.....	8
8 – Conclusões	10

1 - Diagrama Geral da Arquitetura

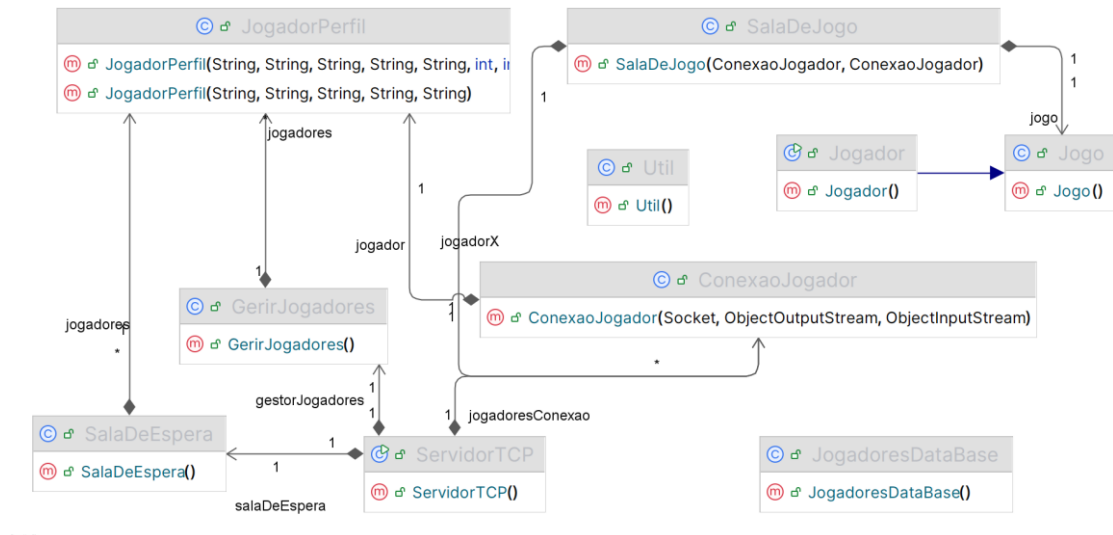


Figura 1: UML simplificado

A estrutura geral do programa manteve-se igual, somente foram adicionadas páginas JSP para a interface visual da WEB.

As páginas adicionadas foram:

- login.jsp
- game.jsp
- quadro_honra.jsp
- settings.jsp

2 - Protocolos de Transporte

Sem alterações.

3 - Componentes e Funcionalidades

3.1 Servidor TCP (Concorrente)

Sem alterações.

3.2 Jogador Conectado (Concorrente)

Sem alterações.

3.3 Gerir Jogadores (Iterativo)

Sem alterações.

3.4 JogadoresDataBase (Iterativo)

- Foi adicionado um método para criar cópias de segurança do quadro de honra.
- Métodos atualizados com informações adicionais a serem salvas no XML.

3.5 SalaDeEspera (Concorrente)

Sem alterações.

3.6 Jogador Perfil (Iterativo)

- Foi adicionado o parâmetro e métodos para guardar a cor preferida.

3.7 ConexaoJogador (Concorrente)

Sem alterações.

3.8 SalaDeJogo (Concorrente)

- Contagem do tempo de jogada adicionado.
- Adicionada a verificação dos 30 segundos.

3.9 Jogo e Jogador (Iterativo e Concorrente)

- Adicionado construtor no Jogador, para armazenar Socket, ObjectOutputStream e ObjectInputStream. Pois

3.10 Util (Iterativo)

Sem alterações.

4 Arquitetura para a WEB

A aplicação web foi desenvolvida como uma interface alternativa e moderna para interagir com o sistema de jogo, em conformidade com os requisitos do projeto1. A implementação utiliza a tecnologia **JavaServer Pages (JSP)**, que permite a renderização dinâmica de conteúdo HTML no lado do servidor, combinado com scriptlets Java para executar a lógica.

A arquitetura mantém a separação entre a interface do utilizador (as páginas JSP) e a lógica de comunicação com o servidor de jogo pré-existente. Esta comunicação é gerida através de uma infraestrutura baseada em sockets, encapsulada na classe Jogador. A gestão do estado do utilizador entre os diferentes pedidos HTTP, que são inerentemente sem estado, é assegurada pelo mecanismo de **sessão do container de**

servlets (HttpSession), um pilar fundamental para qualquer aplicação web com autenticação.

4.1 Login.jsp

Nas páginas JSP, sempre que um utilizador faz login, é criada uma nova instância da classe Jogador (através do `new Jogador()`). Se o login for bem-sucedido, este objeto é guardado na sessão do utilizador com `session.setAttribute("jogador", jogador)`. Assim, nas interações seguintes, o objeto Jogador é recuperado da sessão e usado para enviar comandos ao servidor (por exemplo, `/desafiar`), além de receber respostas ou atualizações do jogo. Os atributos de socket e streams da classe Jogador foram marcados como **transient**, porque estes objetos não são serializáveis e não devem ser guardados caso a sessão seja persistida em disco.

4.1.1 Funcionalidades da Página de Login (login.jsp)

- **Autenticação:** O utilizador preenche um formulário com o seu nickname e password. Quando clica em "Entrar", a própria página trata o pedido (via POST).
- **Ligação ao Servidor:** O JSP deteta que recebeu um POST, cria uma nova instância de Jogador (que, por sua vez, estabelece a ligação por socket) e envia as credenciais para o servidor confirmar.
- **Gestão da Sessão:** Se o login for validado pelo servidor, a instância Jogador e o nickname são guardados na sessão. Isto garante que o estado da ligação e a identidade do utilizador se mantenham enquanto navega pela aplicação, como na sala de espera ou no jogo.
- **Feedback ao Utilizador:** A página mostra de imediato se o login correu bem ou mal, com mensagens visuais a indicar o que se passou.

4.1.2 Funcionalidades na Sala de Espera

- **Comandos ao Servidor:** Na sala de espera, o utilizador pode escolher entre listar quem está online ou desafiar um adversário. Cada ação manda um comando específico para o servidor usando o objeto Jogador da sessão. A resposta do servidor é mostrada logo na página.
- **Campo de Autocomplete:** Para facilitar o envio de desafios, existe um campo de texto que sugere nomes de jogadores enquanto o utilizador escreve, usando o plugin jQuery UI.
- **Navegação e Entrada no Jogo:** A sala de espera também tem links para outras partes da aplicação, como as "Definições de Perfil" ou o "Quadro de Honra". E, claro, quando um desafio é aceite e o servidor autoriza o jogo a começar, a página redireciona automaticamente para o `game.jsp`, garantindo que a transição para o jogo seja o mais natural possível.

4.2 Game.jsp

A página **game.jsp** é onde o jogo de Gobang acontece.

4.2.1 Implementação e Estrutura

- **Lógica de Servidor (Scriptlets Java):** No topo do ficheiro, há um grande bloco de código Java que gere todo o ciclo de vida do jogo.
- **Apresentação (HTML/CSS):** O HTML desenha o tabuleiro e a interface, enquanto o CSS dá estilo ao jogo, incluindo a possibilidade de mudar a cor de fundo.
- **Interatividade (JavaScript):** No final da página, um bloco de JavaScript melhora a experiência, dando feedback visual imediato aos jogadores.

4.2.2 Gestão do Estado e Comunicação

O funcionamento do jogo depende diretamente do objeto Jogador que está guardado na sessão. É esse objeto que faz a ponte com o servidor. O fluxo segue este ciclo:

- **Verificação de Turno:** A página está sempre a verificar se é a vez do jogador (usando a variável `eMinhaVez`). Se for, cria links clicáveis nas posições vazias do tabuleiro.
- **Comunicação Bloqueante:** Se não for a vez do jogador, a página “trava” num `jogo.getMessage()`, à espera de receber do servidor o próximo estado do jogo (com a jogada do adversário).
- **Atualização do Estado:** Quando chega a jogada do adversário, o tabuleiro é atualizado, `eMinhaVez` passa a `true`, e a página volta a ser renderizada para mostrar o novo estado e permitir a jogada seguinte.
- **Fim de Jogo:** Quando alguém ganha, perde ou empata, o código atualiza a mensagem e bloqueia o tabuleiro para evitar mais jogadas.

4.2.3 Desenho Dinâmico do Tabuleiro

O tabuleiro é redesenhado sempre que a página recarrega. O JSP percorre uma matriz (`int[][]` tabuleiro) que representa o estado atual. Para cada posição, decide o que mostrar:

- Se a casa tem um X ou um O, mostra um div com a classe certa (peça preta ou branca).
- Se a casa está vazia e for a vez do jogador, mostra um link `<a>` que, quando clicado, envia a jogada ao servidor.

4.2.4 Melhorias na Experiência do Jogador (UX)

Para lidar melhor com o facto de ser um sistema síncrono (pedido-resposta), foram feitas duas melhorias em JavaScript:

- **Feedback Visual Imediato:** Para não haver atraso em ver a própria jogada, o JavaScript cria logo uma “peça fantasma” (sem cor total) assim que o jogador clica, antes mesmo de o pedido chegar ao servidor.
- **Bloqueio Temporário:** Ao mesmo tempo, o JavaScript ativa um overlay que cobre todo o tabuleiro, impedindo cliques duplos ou rápidos demais. Só depois de mostrar este feedback visual é que o pedido ao servidor é enviado, tornando a experiência mais suave.

4.3 Páginas de definições e quadro de honra

4.3.1 Settings.jsp

Esta página permite que cada jogador personalize a sua informação. Foi implementada como uma única página JSP que gere tanto a apresentação dos dados atuais como o processamento das alterações.

- **Arquitetura de Apresentação e Processamento:** A página utiliza o método GET para carregar e exibir os dados do perfil do utilizador (nickname, data de nascimento, cor preferida e foto), pré-preenchendo os campos de um formulário HTML. As alterações são submetidas via POST, e a mesma página contém a lógica para processar os novos dados.
- **Gestão de Upload de Ficheiros:** A alteração da fotografia é uma funcionalidade que requer o processamento de dados multipart/form-data. Esta capacidade foi ativada através da configuração do web.xml, que instrui o container de servlets a preparar o pedido para que o objeto jakarta.servlet.http.Part possa ser extraído e processado pelo JSP. O ficheiro recebido é então guardado no servidor com um nome único para evitar conflitos.
- **Persistência e Feedback ao Utilizador:** Após a validação, todos os dados atualizados são guardados no ficheiro Jogadores.xml através da classe JogadoresDataBase. A página implementa o padrão **Post/Redirect/Get**, redirecionando o utilizador para a mesma página após a submissão. Isto previne o reenvio accidental de dados e permite a exibição de uma mensagem de sucesso ou erro, que é temporariamente armazenada na sessão.

4.3.2 Quadro_honra.jsp

O Quadro de Honra funciona como o "Hall da Fama" do jogo, exibindo um ranking dinâmico dos jogadores com base no seu desempenho.

- **Carregamento e Ordenação Complexa:** Ao ser acedida, a página carrega a lista completa de jogadores do Jogadores.xml. A principal complexidade reside na ordenação dos dados, que é realizada em memória através de um Comparador personalizado. Este Comparador implementa a regra definida: ordena primeiro pelo número de vitórias (em ordem decrescente) e, em caso de empate, utiliza o tempo de jogo como critério de desempate (em ordem crescente).
- **Renderização Dinâmica:** A tabela de ranking é gerada dinamicamente com um loop for que percorre a lista já ordenada. Para cada jogador, são exibidos o seu ranking, a fotografia de perfil, o nickname, a bandeira da nacionalidade, o número de vitórias e o tempo médio de jogo.

- **Geração de Cópias de Segurança:** Foi implementada uma funcionalidade de backup. Um botão na página permite ao utilizador acionar um processo no servidor que gera um novo ficheiro XML contendo um "snapshot" do ranking atual. A página JSP, ao detetar este pedido, altera os cabeçalhos da resposta HTTP para forçar o navegador a descarregar o ficheiro gerado, garantindo assim a preservação dos dados históricos.

5 - Estruturas de Dados Persistentes

A estrutura do ficheiro XML manteve-se muito similar à da primeira parte do trabalho, porém foram feitas alterações, para guardar o tempo de jogo e data e foto de maneira mais eficaz, e também foi adicionada a informação para a cor preferida.

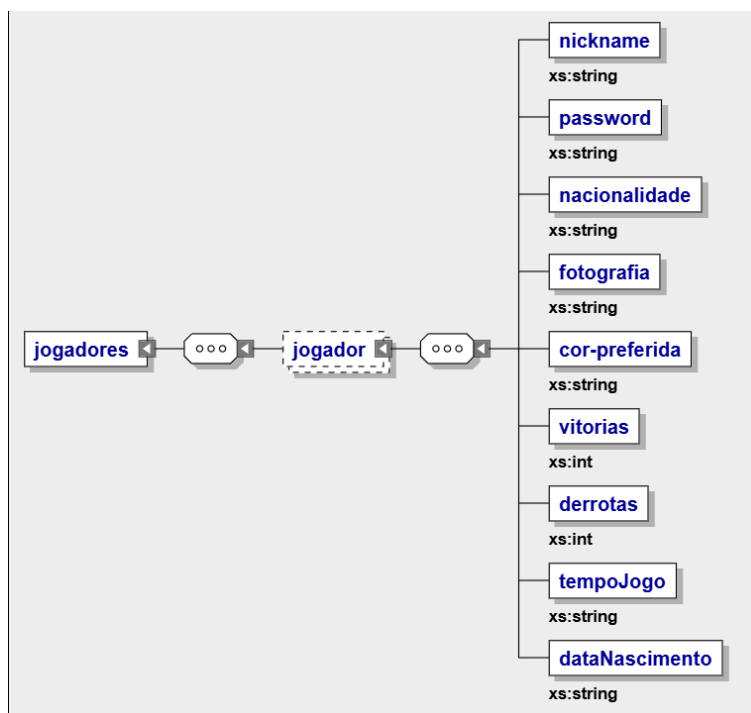
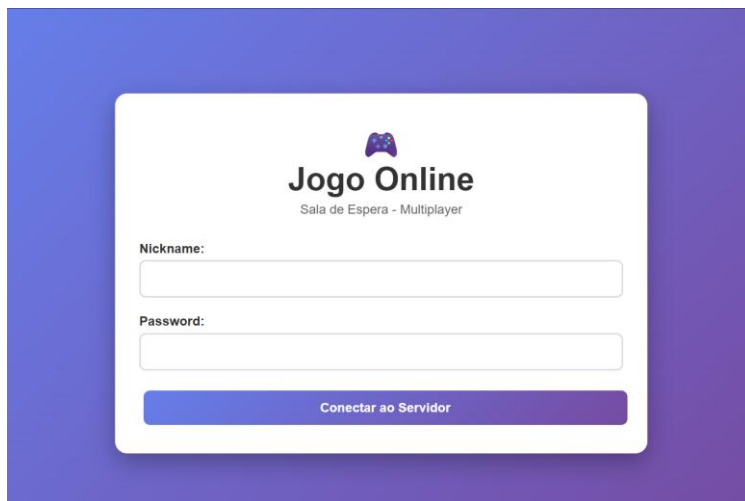


Figura 2: Estrutura do XSD

6 - Protocolo de Aplicação

Sem alterações.

7 – Exemplos da Aplicação



The login screen features a purple gradient background. At the top center is a purple game controller icon. Below it, the text "Jogo Online" is displayed in a large, bold, black font, followed by "Sala de Espera - Multiplayer" in a smaller, regular black font. The form includes two input fields: "Nickname:" and "Password:", each with a white border and a light gray background. Below these fields is a wide, rounded rectangular button with a blue-to-purple gradient, labeled "Conectar ao Servidor" in white text.

Figura 3:Login



The waiting room screen has a purple gradient background. At the top center is a purple game controller icon. Below it, the text "Jogo Online" is displayed in a large, bold, black font, followed by "Sala de Espera - Multiplayer" in a smaller, regular black font. The screen displays a welcome message: "Bem-vindo, Gianni!". Below this are three buttons: "Listar Jogadores" (purple), "Definições" (teal), and "Quadro de Honra" (yellow). There is a text input field labeled "Nome do adversário" and a "Desafiar" button (purple) to its right. Below the input field is a light gray rectangular area containing the number "1". At the bottom left is a red button labeled "Sair" with a white icon.

Figura 4:Sala de Espera

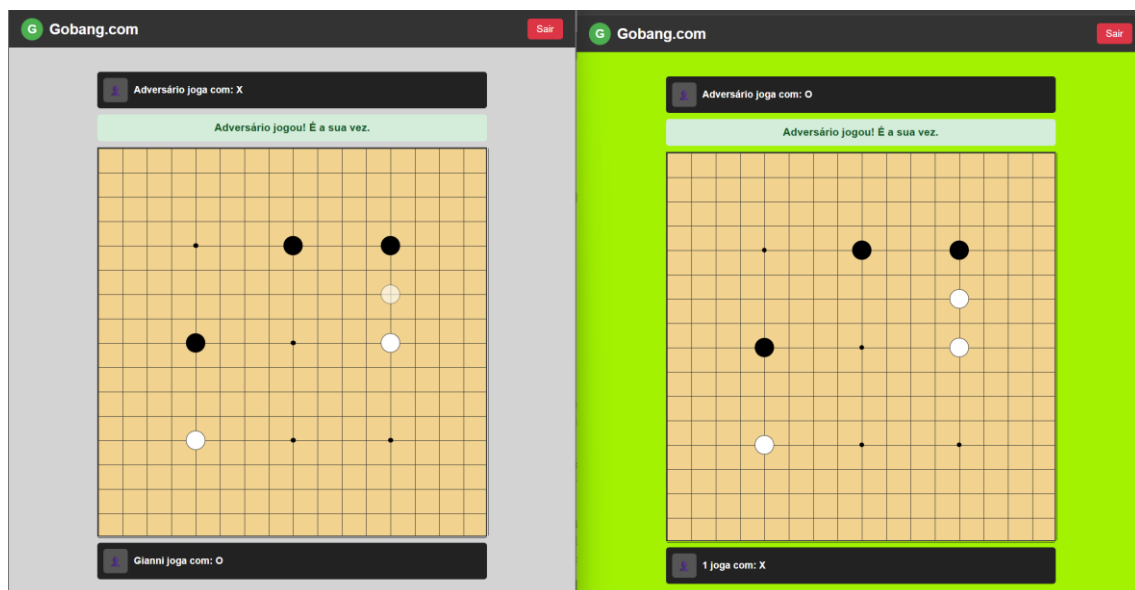


Figura 5:Jogo entre 2 jogadores

QUADRO DE HONRA

#	JOGADOR	VITÓRIAS	TEMPO DE JOGO
1	 1 	35	04:54 min
2	 2 	29	09:42 min
3	 ZéMambinho 	20	05:30 min
4	 ZéMambo 	20	52:29 min
5	 joao298 	6	25:43 min
6	 aninhas298 	3	10:00 min
7	 Gianni 	1	01:14 min

[Salvar Cópia de Segurança \(XML\)](#)

[Voltar à Sala de Espera](#)

Figura 6:Quadro de Honra



Definições de Perfil

Olá, **Gianni**. Altere os seus dados abaixo.



Alterar Foto de Perfil

Choose File No file chosen

Nickname

Gianni

Data de Nascimento

02/25/2005

Cor de Fundo Preferida para o Jogo

Guardar Alterações Voltar

Figura 7: Definições

8 – Conclusões

A arquitetura desenvolvida para o jogo “15 em linha” revela-se eficaz e bem-adaptada ao contexto de aplicações interativas em rede.

A organização modular do código e a estrutura de dados é baseada em XML permitem a adição de novas funcionalidades, como modos de jogo ou sistemas de pontuação mais complexos, sem a necessidade de reestruturar o projeto base.

A segurança é assegurada, em parte, através da encriptação das palavras-passe com o algoritmo SHA-256, protegendo as credenciais armazenadas, embora a ausência de mecanismos mais avançados como encriptação das comunicações ou autenticação por sessão limite a sua robustez em ambientes com maior exposição.

No que toca à transparência, o sistema esconde eficazmente a complexidade da gestão de sessões e persistência de dados do ponto de vista do utilizador, oferecendo uma interação simples e fluida.

No entanto, a tolerância a falhas pode ser considerada uma das limitações da solução, pois a dependência de um único ficheiro XML para a persistência dos dados e a ausência de mecanismos automáticos de recuperação ou backups representam potenciais pontos de falha. Ainda assim, para o âmbito em que se insere, esta

arquitetura constitui uma base sólida, clara e funcional, capaz de oferecer um bom desempenho e de evoluir conforme as necessidades futuras do sistema.

Acreditamos que foi feito um bom trabalho na superação de adversidades encontradas ao longo do desenvolvimento, e concordamos que o desenvolvimento deste projeto teve uma grande contribuição para o nosso conhecimento sobre os temas abordados.